

Generative Robotic Control with Tunable Safety: Flow Matching for Safety-Critical Tasks

Albert Author¹ and Bernard D. Researcher²

Abstract—Sampling-based predictive control (SBPC) algorithms, such as model predictive path integral (MPPI), have become a popular method for robotic motion planning applications due to their computational efficiency on modern computing architectures. However, it is difficult to enforce hard safety constraints for such techniques because they average over samples to realize the final set of control inputs. While individual samples may satisfy a given constraint, the sample average can violate the safety requirement. Similarly, diffusion and flow matching algorithms have become a popular way to generate robot motion trajectories. In this paper, we introduce a novel strategy to learn a conditional generative model that allows for online tuning of a safety parameter. As for sampling-based controllers, it is difficult to enforce hard safety-critical constraints in these methods. To address these limitations, this paper proposes a safety-critical framework for SBPCs and generative trajectory generation, based on the framework of discrete-time control barrier functions (DCBFs). Our DCBF-based method ensures safety during both the sampling and averaging phases of SBPCs, efficiently producing a rich dataset of multi-modal, collision-free behaviors. We then train a conditional flow matching model on these data to learn a generative policy that can be modulated at inference time by a single safety parameter. We demonstrate our approach for cluttered environments for a double integrator and a quadrotor simulation. Our results indicate that the proposed MPPI-DCBF framework consistently achieves superior safety guarantees compared to baseline MPPI methods, together with a context-aware conditional flow matching model, paving the way for real-time, safe SBPC and generative robotic control.

I. INTRODUCTION

Sampling-based predictive control (SBPC) has been gaining traction in the nonlinear optimal control community. Completely general optimal control approaches often suffer from computational complexity associated with solving stochastic Hamilton–Jacobi–Bellman equations backwards in time, commonly referred to as the curse of dimensionality. SBPC is an alternative to gradient-based model predictive control (MPC), where a simple sampling procedure is used in place of a sophisticated nonlinear optimizer [?], [?], [?]. Algorithms in this family include model predictive path integral control (MPPI) [?], predictive sampling [?], and cross-entropy methods [?]. MPPI, for example, circumvents computational complexity by employing forward sampling of stochastic dynamics combined with Monte Carlo approximation techniques [?]. The optimal control is then computed as the exponentially weighted average of the sampled trajectories, enabling real-time implementation. SBPC algorithms are particularly well-suited to recent advances in parallel computing architectures and parallel simulation [?], [?], [?], enabling them to scale to complex problems like in-hand

dexterous manipulation [?], legged locomotion [?], and more [?].

Despite these advantages, SBPCs such as MPPI inherently struggle with safety guarantees, such as collision avoidance. Safety constraints are typically incorporated indirectly through penalty terms in the cost function that are inversely proportional to the robot’s proximity to obstacles and heavily penalize the collision [?], [?]. However, relying solely on these penalties lacks explicit safety guarantees due to the nature of the sampling averaging procedure. This limitation motivates our goal to integrate safety-critical constraints explicitly into the SBPC frameworks. A common method to ensure safety in collision avoidance tasks is to employ a safety filter, which minimally modifies the output of a potentially unsafe nominal controller to guarantee the satisfaction of the constraints [?]. However, a simple safety filter applied *only* to the control output is insufficient to guarantee safety due to the tight coupling between control objectives and long-term safety criteria that are potentially conflicting. Previous works have used control Lyapunov functions (CLF) and control barrier functions (CBF), often with relaxation factors, to solve this issue as an optimization problem, for mediating safety and feasibility [?], [?], or solve the optimization problem in the context of model predictive control (MPC) to take future information into account over a longer horizon [?]. Recent works have encoded this concept through predictive safety filters, which leverage predictive CBFs in layered control architectures [?]. However, in cluttered environments, this direct optimization often becomes non-convex and intractable.

This computational burden motivates a shift from online constrained optimization to learning-based control, where complex planning is performed offline to curate datasets of safe, multi-modal behavior. Generative models, such as diffusion and flow matching, can synthesize motion trajectories from diverse, high-dimensional demonstrations [?], [?]. However, they do not natively provide formal safety guarantees during rollout. Previous work attempts to encode safety by conditioning or regularizing generation [?], [?], but deployments still lack mechanisms to systematically trade off safety and feasibility. This can be critical in real-world tasks where the robot must adjust caution to context.

This paper introduces a supervised learning framework that tightly connects generative modeling with a sampling-based planner, preserving computational efficiency while enabling real-time control at inference time with multi-modalities in obstacle avoidance tasks (see Fig. ??). Our generative policy builds upon generative predictive control

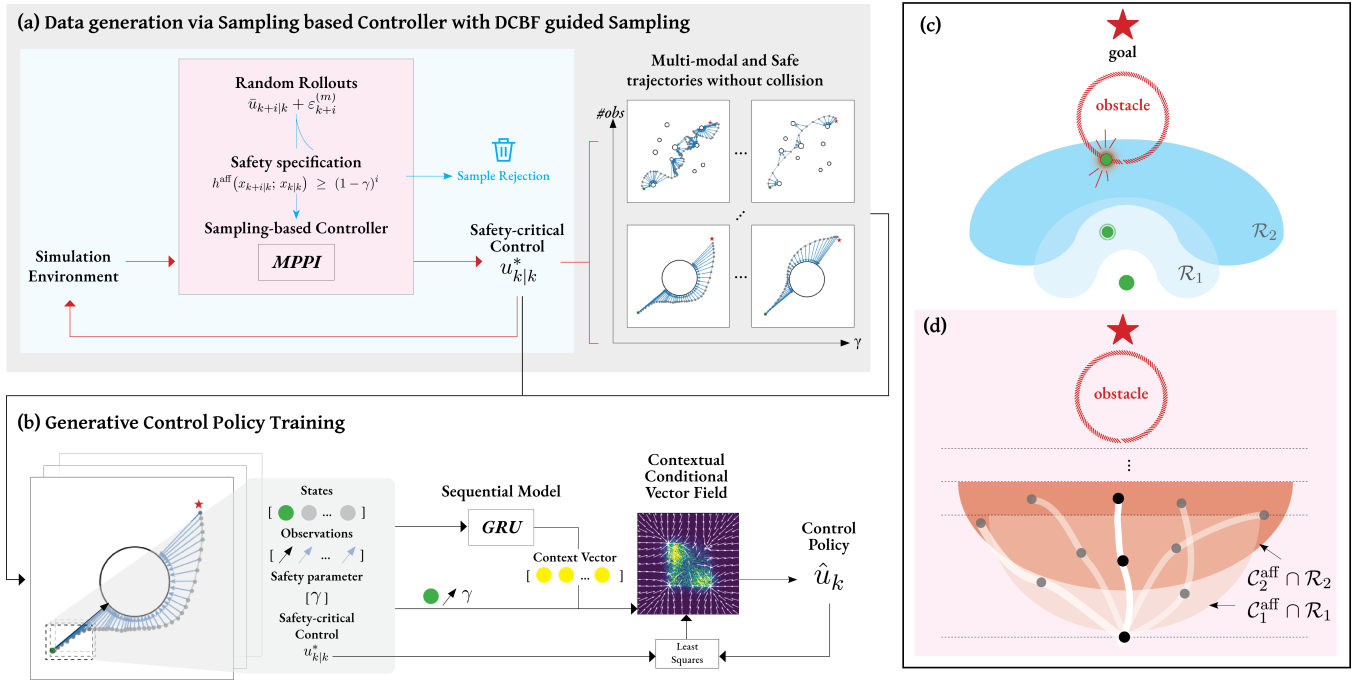


Fig. 1. Overview of our end-to-end pipeline for learning a safety-tunable generative control policy. (a) Our safety-critical planner, MPPI-DCBF, uses an affine discrete-time control barrier function to reject unsafe rollouts. This process generates a rich dataset of multi-modal, collision-free trajectories whose behavior is modulated by the safety parameter, γ . (b) A conditional flow matching model learns a generative policy, where a sequential model generates a context vector to guide the learned vector field, enabling a real-time policy tunable by γ at inference. (c) In sampling-based predictive control like MPPI, the final averaging of rollouts can be unsafe for systems with non-convex reachable sets ($\mathcal{R}_i$). (d) MPPI-DCBF solves this by ensuring a convex reachable subset of the safe set at each step, which also provides a safe warm-start to promote recursive feasibility.

(GPC), which learns a vector field to map noise to control actions conditioned on observations and then applies control in a receding horizon fashion [?]. To achieve this, we introduce two key components. First, we propose a novel predictive planner, MPPI-DCBF, which uses an affine discrete-time control barrier function (DCBF) to filter unsafe rollouts within a sampling-based planner. This method generates a rich dataset where behaviors are implicitly modulated by a tunable safety parameter, γ . Second, we extend the GPC framework by training a conditional flow matching model that uses local geometric information, a sequential context vector, and the safety parameter γ to learn a policy that explicitly modulates the trade-off between safety and feasibility at inference time, resolving ambiguities not addressed in the original GPC implementation.

II. PRELIMINARIES

A. Model Predictive Path Integral control

In discrete time, MPPI solves the following non-convex optimization problem at time step k and state $x(0) \in \mathbb{R}^{n_x}$ over the mean control sequence $\{u_{k+i|k}\} \in \mathbb{R}^{n_u}$ for $i \in \{0, \dots, N-1\}$

$$\min_{\{u_{k+i|k}\}} \mathbb{E} \left[\phi(x_{k+N|k}) + \sum_{i=0}^{N-1} \left(q(x_{k+i|k}) + \frac{1}{2} u_{k+i|k}^\top R u_{k+i|k} \right) \right], \quad (1a)$$

$$\text{s.t. } x_{k+i+1|k} = f(x_{k+i|k}, u_{k+i|k} + \varepsilon_{k+i}), \quad (1b)$$

$$x_{k|k} = x(0), \quad \varepsilon_{k+i} \sim \mathcal{N}(0, \Sigma), \quad (1c)$$

Here, $q: \mathbb{R}^{n_x} \rightarrow \mathbb{R}$ is any running cost, $\phi: \mathbb{R}^{n_x} \rightarrow \mathbb{R}$ is a terminal cost, Σ is the control disturbance variance, $R \in \mathbb{R}^{n_u \times n_u}$ is the control cost weight. For the collision avoidance task, the controller can reason about crashing by setting collision penalties using the signed clearance to the closest obstacle $d_{\min}(x)$ in their running cost function, such as

$$q(x) = (x - x_{\text{goal}})^\top Q (x - x_{\text{goal}}) + \kappa_{\text{soft}} e^{-\alpha d_{\min}(x)} + \kappa_{\text{hard}} \mathbf{1}[d_{\min}(x) < 0]. \quad (2)$$

A critical assumption is that the system is affine in control and noise, and that the noise enters the dynamics via the control channel. The MPPI optimal control sequence is a path-integral approximation of reward-weighted average random variations, which is solved by Monte Carlo methods, centered on the optimal policy sequence at the previous time step [?]. The unexecuted part of the sequence is shifted by one time step and acts as the warm start of the next optimization.

Let this warm start sequence from the previous time step be denoted by $\bar{u}_{k+i|k}$. For rollout $m = 1, \dots, M$ with perturbations $\varepsilon_{k+i}^{(m)} \sim \mathcal{N}(0, \Sigma)$, MPPI measures cost per rollout via forward sampling of random controls and states that can be expressed as

$$\begin{aligned} u_{k+i|k}^{(m)} &= \bar{u}_{k+i|k} + \varepsilon_{k+i}^{(m)}, \\ x_{k+i+1|k}^{(m)} &= f(x_{k+i|k}^{(m)}, u_{k+i|k}^{(m)}). \end{aligned} \quad (3)$$

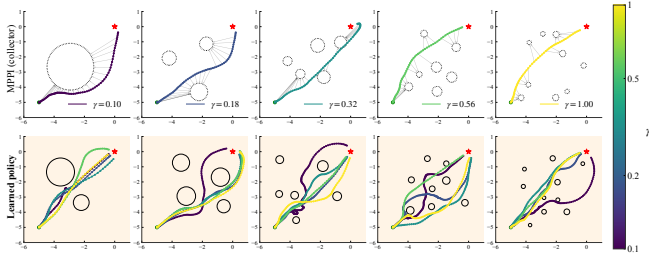


Fig. 2. Collected and generated trajectories under the scalar safety parameter γ . Curve color encodes γ , which induces distinct multi-modalities.

Then, the cost for each rollout is

$$S^{(m)} = \phi(x_{k+N|k}^{(m)}) + \sum_{i=0}^{N-1} \left(q(x_{k+i|k}^{(m)}) + \frac{1}{2} u_{k+i|k}^{(m)\top} R u_{k+i|k}^{(m)} \right). \quad (4)$$

Information-theoretic MPPI uses soft-min importance weights [?]:

$$\omega_m = \exp\left(-\frac{1}{\lambda} [S^{(m)} - \beta]\right), \quad \beta = \min_m S^{(m)}, \quad (5)$$

where $\lambda > 0$ is an inverse temperature which controls how heavily the optimal trajectory is weighted, and β is the minimum cost, used to improve the numerical stability. Then, for $i \in \{0, \dots, N-1\}$, the mean update that gives optimal control sequence is

$$\Delta u_{k+i|k} = \frac{\sum_{m=1}^M \omega_m \varepsilon_{k+i}^{(m)}}{\sum_{m=1}^M \omega_m}, \quad u_{k+i|k}^* \leftarrow \bar{u}_{k+i|k} + \Delta u_{k+i|k}. \quad (6)$$

Then the current optimal control $u_{k|k}^*$ is applied. This parallelizable sampling process makes MPPI highly suitable for solving non-convex optimal control problems on modern GPU hardware. However, a key challenge arises in safety-critical applications. When obstacles are present, a cost for avoiding obstacles does not provide formal safety guarantees, as the final control is averaged over many trajectories.

Several works have focused on incorporating safety into the forward sampling process by shaping the rollout distribution and by injecting feedback, rather than enforcing hard state constraints. For example, the covariance steering method solves this problem by controlling a desired terminal state covariance together with control disturbance variance. This algorithm encourages multi-modal detours around obstacles and avoids collision during the averaging phase [?]. On the other hand, safety-controlled MPPI used importance sampling under embedded barrier state-based feedback control during the forward sampling of MPPI [?]. These methods bias the sampling distribution toward safety, but do not enforce a rigorous safety definition.

B. Flow Matching for trajectory generation

Flow matching is increasingly used in robotics for trajectory generation and policy generation. The trajectory generation process learns a mapping from noise to entire collision-free trajectories by end-to-end supervised learning,

but it can struggle to generalize to out-of-distribution environments and may require complete retraining [?], [?]. In contrast, (conditional) flow matching-based policy generation is more flexible, focuses on learning a mapping to a lower-dimensional control action, conditioned on observations [?].

For collision avoidance, it is essential to generate a policy conditioned on geometric observations while benefiting from offline flow matching training and fast computation from SBPC. Here, we describe the important concept of conditional flow matching. For purposes of clarity, $u_t(\cdot)$ denotes the flow matching vector field (distinct from the MPPI control sequence u). We use $z \in \mathbb{R}^{n_u}$ for a target optimal control policy, *i.e.*, the symbol $u_{k|k}^*$ used in the previous section.

Flow matching aims to learn a marginal vector field $u_t^*(x)$ that converts a simple distribution $x_0 \sim p_0$, such as a standard isotropic Gaussian, into a complex data distribution $z \sim p_{data}$. This is done by considering the marginal probability path $p_t(x)$ that evolves from p_0 to p_{data} . Then, by simulating an ordinary differential equation (ODE) characterized by a marginal vector field, we can generate a meaningful sample.

However, due to the intractable nature of the data distribution, one can only realize a conditional probability path $p_t(x|z)$ rather than $p_t(x)$. We then use a pre-defined conditional vector field $u_t^*(x|z)$ for training to approximate the ground truth ODE (*i.e.*, marginal vector field $u_t^*(x)$) in a supervised regression fashion. This holds since the marginal vector field and the conditional vector field share the same minimizer [?], [?]. Moreover, a marginal vector field can be reconstructed after training via marginalization

$$u_t^*(x) = \int u_t^*(x|z) \frac{p_t(x|z)p_{data}(z)}{p_t(x)} dz, \quad (7)$$

and be used for simulating an ODE to generate samples that adhere to the data. With an optimal transport probability path that drives noise ϵ to z with constant direction and speed in time [?], the conditional flow matching (CFM) model $u_t^\theta(x)$ learns the conditional vector field $u_t^*(x|z)$ per sampled point x from conditional vector field by minimizing the mean-squared loss:

$$\begin{aligned} \mathcal{L}(\theta) &= \mathbb{E}_{t \sim \text{Unif}[0,1], \substack{z \sim p_{data}, \\ x \sim p_t(\cdot|z)}} \left[\|u_t^\theta(x) - u_t^*(x|z)\|^2 \right] \\ &= \mathbb{E}_{t \sim \text{Unif}[0,1], \substack{\epsilon \sim \mathcal{N}(0, I_d), \\ z \sim p_{data}}} \left[\|u_t^\theta(tz + (1-t)\epsilon) - (z - \epsilon)\|^2 \right]. \end{aligned} \quad (8)$$

Here, d is the dimension of the noise ϵ and data z . At inference time, we sample from a simple distribution and simulate a learned marginal vector field to generate data. Flow matching for trajectory generation can be achieved through the conditional flow matching model, where the conditioning is on arbitrary variables or observations, such as a safety parameter and the distance to the obstacle.

To generate a policy using the CFM model, we first assume there is a ground-truth marginal vector field that transforms entire noise into an optimal policy given observations. In our case, we view the task of generating safe robot motion trajectories

as not merely to generate any valid guess, but to generate a control policy conditioned on local geometric observations and states o , a safety parameter γ , and a context vector c , as described in Fig. ??(b). In this case, we need to expand the expectation from (??) over all possible o , γ , and c . Thus, we define our training objective as

$$\mathcal{L}(\theta) = \mathbb{E}_{\substack{t \sim \text{Unif}[0,1], \\ \epsilon \sim \mathcal{N}(0, I_d), \\ (z, o, \gamma, c) \sim p_{\text{data}}(z, o, \gamma, c), \\ x \sim p_t(\cdot|z)}} \left[\|u_t^\theta(x|o, \gamma, c) - (z - \epsilon)\|^2 \right]. \quad (9)$$

C. Discrete-Time Control Barrier Functions

Let $\mathcal{C} = \{x \in \mathbb{R}^n : h(x) \geq 0\}$ denote the safe set for a continuously differentiable h . For the discrete-time control-affine system

$$x_{k+1} = f(x_k) + g(x_k)u_k, \quad (10)$$

a function h is a discrete-time control barrier function (DCBF) associated with the control system (??) if there exists $\gamma \in (0, 1]$ such that for all x_k there exists a control u_k with

$$\Delta h(x_k, u_k) := h(x_{k+1}) - h(x_k) \geq -\gamma h(x_k). \quad (11)$$

Condition (??) implies the one-step decay bound

$$h(x_{k+1}) \geq (1 - \gamma) h(x_k) \implies h(x_k) \geq (1 - \gamma)^k h(x_0),$$

i.e., the lower bound on the barrier value decays exponentially with base $(1 - \gamma)$, ensuring forward invariance of $\mathcal{C}$ [?], [?].

To be compatible with the averaging in MPPI, we use an affine DCBF derived from a supporting hyperplane to the obstacle boundary. For a convex obstacle, let p be the point on its boundary closest to the current state $x_{k|k}$, with outward normal n . We define the control barrier function as

$$h^{\text{aff}}(x_{k+i|k}; x_{k|k}) := \frac{n^\top (x_{k+i|k} - p)}{n^\top (x_{k|k} - p)}. \quad (12)$$

The safe set for each i -th step is defined as

$$\mathcal{C}_i^{\text{aff}} = \{x \in \mathbb{R}^{n_x} \mid h^{\text{aff}}(x_{k+i|k}; x_{k|k}) \geq 0\}, \quad (13)$$

is a half-space, which is convex. Applying the DCBF condition (??) to this affine barrier function provides our safety specification for each rollout:

$$h^{\text{aff}}(x_{k+i+1|k}; x_{k|k}) \geq (1 - \gamma) h^{\text{aff}}(x_{k+i|k}; x_{k|k}). \quad (14)$$

Iterating this condition from $i = 0$ with $h^{\text{aff}}(x_{k|k}; x_{k|k}) = 1$ yields an exponentially decaying lower bound on the safety margin along the prediction horizon:

$$h^{\text{aff}}(x_{k+i|k}; x_{k|k}) \geq (1 - \gamma)^i, \quad i = 0, \dots, H. \quad (15)$$

III. PROBLEM DEFINITION

We consider a robot whose dynamics are described by the discrete-time control affine system in (??), evolving in a cluttered environment with static obstacles. This work has two primary objectives. First, we experimentally demonstrate that our proposed controller, MPPI-DCBF, achieves superior safety guarantees over a standard baseline MPPI with different choices of the safety parameter γ . We validate this approach and its scalability on both LTI systems and complex nonlinear models, such as a quadrotor. Second, we address the challenge of creating a real-time generative policy that can modulate the trade-off between safety and feasibility by continuously scheduling γ , based on observations.

IV. METHOD

Having presented the necessary background of model predictive path integral control, discrete-time barrier functions, and conditional flow matching, we now formulate the proposed framework.

A. Ego-centric barrier function for multi-obstacle avoidance

We define a *nominal polytope* and associated level sets to account for multiple obstacles in the environment. Let $q_{k+i|i} \in \mathbb{R}^d$ denote the robot position at state $x_{k+i|i}$ at horizon $i = 0, \dots, H - 1$. We assume that the robot only can sense obstacles within a finite radial distance R , and that the obstacles within that region is denoted by $\mathcal{O}_j(q_{k+i|i})$. Within a sensing range R , we define the polytope $\mathcal{P}$ that is tangent to each obstacle j and level sets H_p inside, that is the minimum of $h_j^{\text{aff}}(x_{k+i|k}; x_{k|k}) \geq (1 - \gamma)^i$. $i = 0, \dots, H$. (16)

This ensures that the robot maintains a safe distance from all obstacles throughout the prediction horizon.

B. The MPPI-DCBF Algorithm

Our proposed algorithm, model predictive path integral control with DCBF-based sample rejection (MPPI-DCBF), modifies the standard MPPI framework to formally enforce safety. The core of our method is the safety specification in (??), which is designed to guarantee the forward invariance of the sequence of time-varying safe sets $\mathcal{C}_i^{\text{aff}}$ defined in (??). This is achieved by integrating the DCBF condition directly into the sampling step of the MPPI algorithm. Any sampled rollout that violates the condition (??) is rejected by assigning infinite cost to that rollout. Note that in the importance weighting of MPPI (??), an infinite cost is the same as a sample rejection. Consequently, the optimal control sequence (??) is computed as a convex combination of only those rollouts that are certified to be safe. The complete procedure is detailed in Algorithm ??.

While the proper design of terminal ingredients (i.e., a terminal set and cost) can formally guarantee recursive feasibility in traditional MPC, the non-convex nature of the resulting optimization problem often makes real-time implementation computationally intractable [?]. SBPC methods such as MPPI face similar challenges. Rather than proving recursive feasibility, our approach ensures safety by

enforcing DCBF constraints at each time step, which relies on the weaker but more practical condition of pointwise feasibility, formalized in the following assumption.

Assumption 1: For the discrete-time control affine system (??) and for any initial state $x_{k|k} \in \mathcal{C}_i^{\text{aff}}$, there exists at least one control sequence $\{u_{k+i|k}\}_{i=0}^{N-1}$ such that the resulting trajectory $\{x_{k+i|k}\}_{i=1}^N$ lies within the sequence of safe sets defined in (??), i.e., $x_{k+i|k} \in \mathcal{C}_i^{\text{aff}}$ for all $i \in \{1, \dots, N\}$.

Assumption 1 is the condition for pointwise feasibility of the MPPI-DCBF algorithm at the current state; it ensures that the set of valid trajectories for the planner to find is non-empty. It is important to note that the parameter γ governs the trade-off between safety and feasibility. Although we assume there exists a valid trajectory, a smaller γ yields more conservative (safer) behavior, but results in a smaller safe set (??). Conversely, a larger γ expands the safe set, making it easier to find a valid trajectory, but may result in safety violations in practice if modeling assumptions are violated.

Under the assumption of pointwise feasibility, we can formally analyze the properties of the MPPI-DCBF averaging process.

Proposition 1: For the discrete-time control affine system (??), if all contributing rollouts (m) satisfy $x_{k+1|k}^{(m)} \in \mathcal{C}_1^{\text{aff}}$, then the averaged state $x_{k+1|k}^*$ under the MPPI-DCBF control is also guaranteed to be in the safe set, i.e., $x_{k+1|k}^* \in \mathcal{C}_1^{\text{aff}}$.

Proof: Under the control affine dynamics, the state $x_{k+1|k}^{(m)}$ in the next time step is an affine function of the control inputs. Since the admissible inputs are a convex polytope and the control update of MPPI is a convex combination of control perturbations (??), the resulting state $x_{k+1|k}^*$ is a convex combination of the rollout states:

$$x_{k+1|k}^* = \sum_m \hat{\omega}_m x_{k+1|k}^{(m)},$$

where $\hat{\omega}_m \in \mathbb{R}_{>0}$ and $\sum_m \hat{\omega}_m = 1$ are the importance weights derived from (??) after the rejection of unsafe samples. Moreover, since the safe set $\mathcal{C}_1^{\text{aff}}$ is a convex half-space, any convex combination of points within it must also lie within the set. Thus, if $x_{k+1|k}^{(m)} \in \mathcal{C}_1^{\text{aff}}$ for all m , it follows that $x_{k+1|k}^* \in \mathcal{C}_1^{\text{aff}}$. ■

If the dynamics of the system of interest are linear time invariant (LTI) as

$$x_{k+1} = Ax_k + Bu_k, \quad (17)$$

then we can make a similar statement about safety along the *entire* trajectory, rather than just a single step in the MPPI averaging process.

Proposition 2: For the discrete LTI system (??) under Assumption ??, if all contributing rollouts (m) satisfy $x_{k+i|k}^{(m)} \in \mathcal{C}_i^{\text{aff}}$ for all steps $i \in \{1, \dots, N\}$, then the entire averaged MPPI trajectory also remains within the sequence of safe sets, i.e., $x_{k+i|k}^* \in \mathcal{C}_i^{\text{aff}}$ for all i .

Proof: The proof extends the logic from ?? to the full horizon. For an LTI system, the state at any future step i is

an affine function of the control sequence:

$$x_{k+i} = A^i x_k + \sum_{j=0}^{i-1} A^{i-1-j} B u_{k+j}.$$

Thus, the averaged state is a convex combination of the individual rollout states at that same step:

$$x_{k+i|k}^* = \sum_m \hat{\omega}_m x_{k+i|k}^{(m)}, \quad \forall i \in \{1, \dots, N\},$$

where $\hat{\omega}_m$ are the normalized importance weights. Because $\forall i \in \{1, \dots, N\}$, $x_{k+i|k}^{(m)} \in \mathcal{C}_i^{\text{aff}}$ and $\mathcal{C}_i^{\text{aff}}$ is a convex set, the convex combination, $x_{k+i|k}^*$, must also lie within $\mathcal{C}_i^{\text{aff}}$. ■

?? holds to ensure the full-horizon safety of the averaged plan as illustrated in Fig. ??(d). Despite the fact that only the current optimal control $u_{k|k}^*$ is applied, the full-horizon safety of the optimal control sequence is critical for the subsequent step. The remaining portion of the safe trajectory provides a warm start for the next optimization, anchoring the sampling distribution in a safe region of the state space. Our approach in leveraging DCBFs in a model predictive fashion draws a parallel to optimization-based methods, such as MPC-CBF [?].

C. Conditional Flow Matching for Safety-tunable Policy Generation

To generate a policy that can modulate its behavior in real time, we train a generative model on the dataset from our MPPI-DCBF planner. As shown in Fig. ??(b), our model consists of a sequential encoder and a contextual conditional flow matching (CFM) head. The encoder, a gated recurrent unit (GRU), processes a history of states, observations, and controls to produce a context vector c_t . This context allows the model to capture long-term dependencies, which is critical for resolving ambiguities such as deciding which side of an obstacle to navigate around (see Fig. ??). The CFM head then learns a vector field conditioned on this context, the current state and observation, together denoted by o_t , and the desired safety level γ_t to produce the ground truth control action $u_{t|t}^*$ by minimizing the conditional flow matching loss in (??). The complete training and inference procedure is detailed in Algorithm ??, yielding a real-time policy that can be tuned by adjusting the input parameter $\gamma(k)$ without any online sampling.

V. VERIFIER-GUIDED FLOW EXPANSION

In this section, we introduce the verifier for generative policy to update the generative policy and to generate out-of-distribution samples. Here, we extend the notion Sample-rejection-based safe MPPI

$$\begin{aligned} & \max_{\{a_i, m_i\}} \sum_i w_i m_i \\ & \text{s.t. } a_i^\top (q_t - c) \leq (1 - (1 - \gamma)^t) m_i, \quad t = 1, \dots, H, \\ & \quad r_i \|a_i\|_2 \leq a_i^\top (o_i - c) - m_i, \quad \|a_i\|_2 \leq 1, \quad m_i \geq m_{\min}. \end{aligned} \quad (18)$$

Let $c = q_0 \in \mathbb{R}^d$ denote the current robot position in the free space $\mathcal{Q}$ and let $\{q_t\}_{t=0}^H$ be a candidate local

trajectory submitted to verifier, with $p_t := q_t - c$. For a barrier parameter $\gamma \in (0, 1]$, define $\alpha_t = (1 - \gamma)^t$ and $\beta_t = 1 - \alpha_t$. We assumed a finite circular sensing region. Each sensed circular obstacle is represented as

$$\mathcal{O}_i = \{x \in \mathbb{R}^d : \|x - o_i\|_2 \leq r_i\},$$

where o_i and r_i are its center and radius. Instead of fixing the obstacle face normal as in the nominal radial polytope, we choose a variable face

$$a_i^\top (x - c) \leq m_i,$$

where $a_i \in \mathbb{R}^d$ is the verifier face normal and $m_i \in \mathbb{R}$ is its robot-centered margin.

For each sensed obstacle i , the verifier solves the per-face conic problem

$$\begin{aligned} \max_{a_i, m_i} \quad & m_i \\ \text{s.t.} \quad & a_i^\top p_t \leq \beta_t m_i, \quad t = 1, \dots, H, \\ & r_i \|a_i\|_2 \leq a_i^\top (o_i - c) - m_i, \\ & \|a_i\|_2 \leq 1, \\ & m_i \geq m_{\min}. \end{aligned} \quad (P_i)$$

The first constraint is the rollout level-set condition $H_i(q_t) \geq \alpha_t$, since $H_i(q_t) = (m_i - a_i^\top p_t)/m_i$. The second constraint separates the entire disk, because $\min_{x \in \mathcal{O}_i} a_i^\top (x - c) = a_i^\top (o_i - c) - r_i \|a_i\|_2$. The norm bound $\|a_i\|_2 \leq 1$ is the convex replacement of the nonconvex unit-normal constraint, and $m_i \geq m_{\min}$ prevents degenerate faces.

Problem (P_i) is an SOCP because its objective is linear, its rollout and margin constraints are affine, and the disk-separation and normal-bound constraints are second-order cone constraints. After solving all per-obstacle faces, the verifier polytope is

$$\mathcal{P} = \bigcap_i \{x : a_i^\top (x - c) \leq m_i\},$$

or equivalently $Ax \leq b$ with $A_i = a_i^\top$ and $b_i = a_i^\top c + m_i$.

Dohyun: Why ego-centric safety function?

Pre trained flow matching learns the training distribution based on the distributions provided by a planner, while flow expansion deliberately samples outside the current distribution using uncertainty, verifies the samples, adds successful ones to a buffer and retrains during an offline training. Therefore, the method not just generating trajectories, but expanding the support of the learned policy using a verifier-guided loop.

Dohyun: If we present the system as a reduced order model, the real robot is still goverend by a more complicated full-order system. So we need to be clear about the modeling gap. Later safety claims depend on how well the reduced-order trajectory can be tracked by the full-order system. We could add this uncertainty later, but it should be acknowledged.

Dohyun: For a double-integrator system, safety cannot be certified only from position-level containment.

We viewed the verifier as a mathematical indicator function to check if these conditions are met:

- 1) multi-step safety based on /textit{verifier} polytope,
- 2) progress toward a terminal state.

In this way, the validity of sampled control policies are evaluated with respect to these conditions both in global trajectory level in terms of multi-step safety and also local performance.

In the demo, we check the multi-step safety by verifying the existence of a valid *verifier polytope*.

Also, we set a true progress as a monotone right/up staircase, but it could be a more complicated function of the state. If states remain within the same occupancy grid, we also say this is a valid progress, otherwise we will bias the trajectory always crossing the grid cells, which may not be optimal every time.

We discretized the task space into $5 * 5$ occupancy grid cells, and we define the coverage metric as one trajectory covers one path with a monotone lattice path out of all possible paths (i.e., $10C5 = 252$).

Therefore, coverage measures whether the learned generative policy has discovered a global valid trajectory modes beyond the pre-trained data given by the potentially myopic and conservative planner. While validity measures the local performance of the policy with respect to the safety and performance.

A. Verifier Polytope Construction

The existence of a valid *verifier polytope* ensures that the trajectory respects multi-step safety. This polytope is constructed based on the states at each step in the trajectory, and its validity is checked at each time step.

The optimization problem is solved for each nearby obstacle, and the resulting polytopes are intersected to form the final *verifier polytope* as shown

Dohyun: Draw an example where we only verify based on safety, not performance

VI. NUMERICAL EXAMPLES

In this section, we demonstrate the effectiveness of the proposed framework using two simulation examples. During the experiments, all data generation and model training were performed on a workstation equipped with an NVIDIA H100 NVL GPU.

A. 2D Double Integrator model

Consider a linear discrete-time 2D double integrator

$$x_{k+1} = Ax_k + Bu_k. \quad (19)$$

To generate a dataset for policy learning, we first employ our proposed MPPI-DCBF planner (Algorithm ??) with a sampling time of $\Delta t = 0.1s$. The environment consists of a fixed start at $(-5, -5)$ and a goal at $(0, 0)$, populated

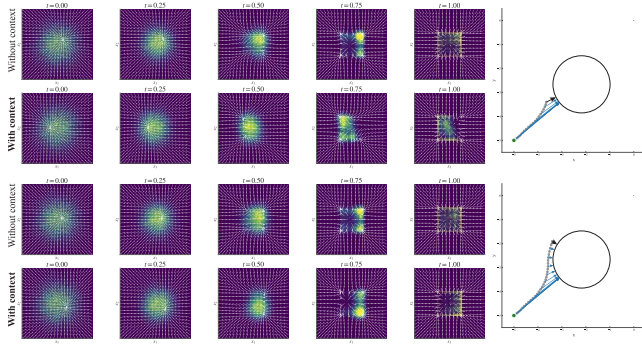


Fig. 3. Flow matching transforms a simple noise distribution into a target data distribution of safe control policies. **Without context**, the learned vector field averages all behaviors into a single mode, leading to a greedy, goal-seeking policy (note the concentrated, goal-directed samples at $t = 0.75$). **With context**, the model learns to distinguish between different situations (e.g., passing an obstacle on the left vs. right), enabling it to generate multi-modal, detouring maneuvers with 1 to 10 randomly positioned circular obstacles. This domain randomization creates a challenging scenario where a single set of tuned cost penalties for a baseline planner is unlikely to generalize effectively across all configurations. Our planner produces a rich dataset of multi-modal collision-free trajectories by varying the safety parameter $\gamma \in (0, 1]$. As shown in the top row of Fig. ??, lower values of γ lead to more conservative, wide-berth maneuvers, while higher values produce more aggressive maneuvers with tighter obstacle clearance.

Before training a policy, we benchmark the performance of our MPPI-DCBF planner against a baseline MPPI controller for the single obstacle case, with results summarized in Table ?. Both controllers solve the optimal control problem in (??) with identical quadratic cost weights ($Q_p = 100I_2$, $Q_v = 10I_2$, $R = 0.05I_2$, $P_p = 1000I_2$, and $P_v = 100I_2$, where the subscripts p and v denote position and velocity, respectively). The baseline MPPI uses an additional penalty-based running cost as in (??), $q_{\text{obs}}(x) = 5 \cdot 10^4 \cdot \mathbf{1}[d_{\min}(x) < 0] + 100e^{-d_{\min}(x)}$. Analysis of the results reveals several key insights. Most notably, our MPPI-DCBF consistently achieves superior safety guarantees compared to the baseline MPPI method, even at a shorter prediction horizon with lower control cost. Importantly, this comes at a minor computational overhead (2-3 ms per step), indicating that safety is achieved without sacrificing performance.

We learn these multi-modal behaviors by training the conditional flow matching model, where the bottom row of Fig. ?? demonstrates that the generative policy successfully reproduces these distinct modalities for different fixed values of γ . Since the model learns a continuous mapping, it can interpolate between its distinct behaviors seen during training. This enables the synthesis of novel control strategies by implementing a dynamic schedule for $\gamma(k)$ at inference time. Here, we design a heuristic schedule:

$$\gamma(k) = \gamma_{\min} + (\gamma_{\max} - \gamma_{\min}) \left(1 - e^{-\beta d(k)}\right) e^{-\alpha \max\{0, v(k)\}}, \quad (20)$$

where $d(k)$ is the distance to the nearest obstacle, $v(k)$ is the relative velocity towards it, and sensitivity parameters

TABLE I
MPPI [?] AND MPPI-DCBF BENCHMARK IN TERMS OF COMPUTATION TIME, SUCCESS RATE, MINIMUM CLEARANCE, AND CONTROL COST INTEGRAL. N DENOTES THE PREDICTION HORIZON FOR BOTH ALGORITHMS.

$N = 7$	Comp. time [ms]	Success rate [%]	Minimum distance [m]	Cost Integral
MPPI	3.6±0.0	61.6	0.12±0.22	1.15±0.29
MPPI-DCBF ($\gamma=0.1$)	5.8±0.0	100.0	0.65±0.08	1.16±0.24
MPPI-DCBF ($\gamma=0.2$)	5.9±0.0	100.0	0.40±0.11	1.10±0.27
MPPI-DCBF ($\gamma=1.0$)	5.8±0.0	100.0	0.22±0.15	1.04±0.27
$N = 11$	Comp. time [ms]	Success rate [%]	Minimum distance [m]	Cost Integral
MPPI	5.3±0.0	100.0	0.14±0.09	1.91±0.19
MPPI-DCBF ($\gamma=0.1$)	8.9±0.0	100.0	0.98±0.06	1.82±0.11
MPPI-DCBF ($\gamma=0.2$)	8.9±0.0	100.0	0.66±0.05	1.77±0.11
MPPI-DCBF ($\gamma=1.0$)	8.9±0.0	100.0	0.42±0.06	1.77±0.15

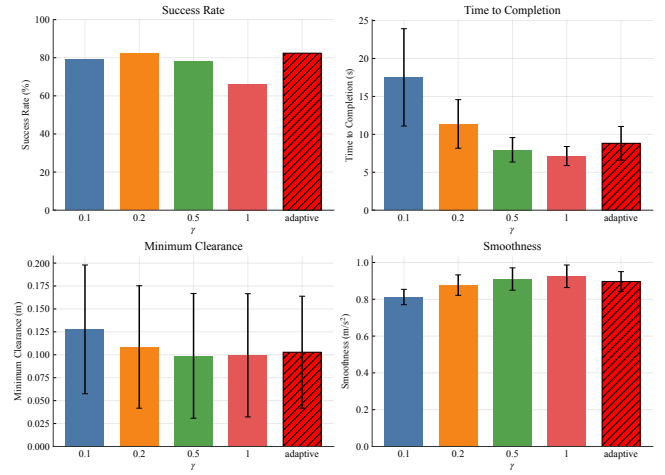


Fig. 4. Performance of the learned generative policy over 1000 randomized trials across four key metrics. The first four bars in each plot correspond to policies using a fixed safety parameter ($\gamma \in \{0.1, 0.2, 0.5, 1.0\}$). The final hatched bar shows the performance of our adaptive schedule from (??).

α, β were fine-tuned to be 0.136, 1.151, respectively. This schedule is designed to increase conservatism (i.e., lowers γ) as the robot approaches an obstacle. Fig. ?? summarizes the performance of the policy over 1000 randomized trials, showing that our adaptive policy achieves a strong balance between safety and feasibility in terms of minimum clearance and success rate. An illustrative example of this behavior in a cluttered environment is shown in Fig. ?. However, learning an optimal schedule remains an open problem.

B. Quadrotor

To demonstrate the scalability of our data generation approach to more complex, nonlinear systems, we apply MPPI-DCBF to a simulated quadrotor. The dynamics follow

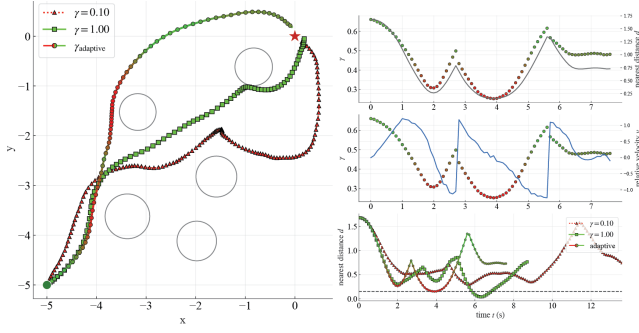


Fig. 5. Trajectories generated by the learned generative policy in a single environment under fixed ($\gamma = 0.1, 1.0$) and our adaptive γ schedule from (??).

[?], augmented with second-order motor dynamics:

$$\frac{d}{dt} \begin{bmatrix} \mathbf{p} \\ \mathbf{v} \\ \mathbf{q} \\ \boldsymbol{\omega} \\ \mathbf{w} \\ \dot{\mathbf{w}} \end{bmatrix} = \begin{bmatrix} -g \mathbf{e}_3 + \frac{1}{m} \mathbf{R}(\mathbf{q}) \mathbf{e}_3 \mathbf{T}(\mathbf{w}) \\ \frac{1}{2} \mathbf{q} \otimes (0, \boldsymbol{\omega}) \\ -\mathbf{J}^{-1}(\boldsymbol{\omega} \times \mathbf{J} \boldsymbol{\omega}) + \mathbf{J}^{-1} \mathbf{M}(\mathbf{w}) \\ \dot{\mathbf{w}} \\ -\frac{2\zeta}{\tau} \dot{\mathbf{w}} - \frac{1}{\tau^2} \mathbf{w} \end{bmatrix} + \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \mathbf{0} \\ \mathbf{0} \\ \mathbf{0} \\ \mathbf{I}_4 \end{bmatrix} \mathbf{u}. \quad (21)$$

where $\mathbf{p} = (x, y, z) \in \mathbb{R}^3$ is the quadrotor's center of mass in the inertial frame, $\mathbf{v} \in \mathbb{R}^3$ is its linear velocity, $\mathbf{q} \in \mathbb{S}^3$ is a quaternion representing body orientation, $\mathbf{R}(\mathbf{q}) \in SO(3)$ is the rotation matrix corresponding to $\mathbf{q}$, $\boldsymbol{\omega} \in \mathbb{R}^3$ is the body angular velocity, m is the total quadrotor mass, g is the gravitational constant, and $\mathbf{J}$ is the matrix of inertia of the body, $\mathbf{e}_3$ is the inertial axis z . Motor dynamics are modeled as a second-order system with damping ratio ζ and time constant τ . With proper scaling, we assume that the input command $\mathbf{u}$ drives the dynamics of the four rotor speeds $\mathbf{w} \in \mathbb{R}^4$. The total thrust $T(\mathbf{w})$, and body moments $\mathbf{M}(\mathbf{w}) \in \mathbb{R}^3$, are functions of the resulting rotor speeds.

Given the high dimensionality and nonlinearity of the system, we employ the one-step predictive safety check mode of our algorithm. In this model, the input command affects the system only through a linear second-order motor model. Since the thrust and moments are evaluated as a function of rotor speed, the discrete dynamics obtained from Euler discretization of the original dynamics are control affine in the current command for a single step. Therefore, the resulting state from the MPPI-DCBF controller remains safe for the next step, as established in Proposition ???. Our planner effectively navigates the quadrotor through a dense field of spherical obstacles, as shown in Fig. ??, demonstrating that our core principle can be extended to challenging, nonlinear systems.

VII. CONCLUSION

This paper presented a framework for creating safety-tunable generative control policies for safety-critical tasks. We introduced a novel MPPI-DCBF planner that uses DCBFs to generate a rich dataset of provably safe multi-modal trajectories for linear systems. By training a conditional flow

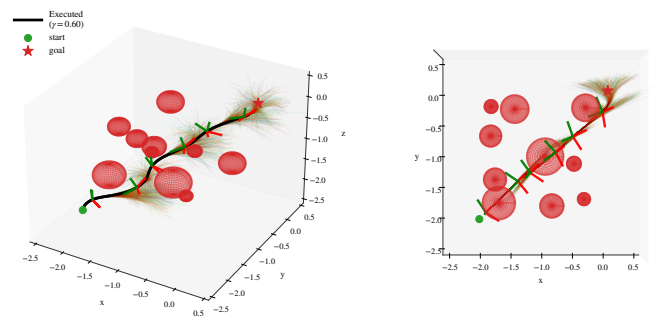


Fig. 6. Demonstration of the MPPI-DCBF planner on a quadrotor (??) navigating a cluttered 3D environment.

matching model on this data, we produced a real-time policy that can modulate its safety and feasibility via a single parameter at inference time, without the need for online sampling. Our experiments on 2D and 3D systems validate the approach, demonstrating a practical method for deploying safe, adaptable, and high-performance learned controllers.

A limitation of our current generative model is that it inherits safety statistically from the data and lacks its own formal certificate. Future work could explore combining our generative policy with a lightweight run-time safety filter to provide hard guarantees. Additionally, learning an optimal γ schedule directly from data remains an interesting open problem.